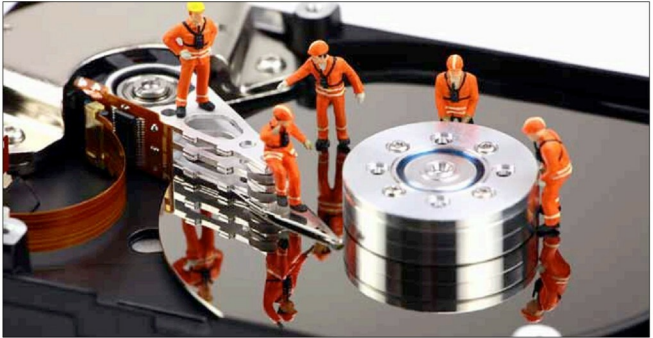


# The Semester Project—Part VII

## The File System in Action

This article, which is part of the series on Linux device drivers, gets the complete SIMULA file system module in action, with a real hardware partition on your pen drive.



Let's first take a look at real SFS.

The code available from [http://linuxforu.com/article\\_source\\_code/nov12/rfs\\_code.tar.bz2](http://linuxforu.com/article_source_code/nov12/rfs_code.tar.bz2) gets to the final tested implementation of Pugs' and Shweta's final semester project. It contains the following:

- *real\_sfs.c*—the code of the earlier *real\_sfs\_minimal.c*, plus the remaining real SIMULA file system functionalities.
- *real\_sfs\_ops.c* & *real\_sfs\_ops.h*—the earlier code plus the additional operations needed for the enhanced *real\_sfs.c* implementations.
- *real\_sfs\_ds.h*—almost the same file as in the previous article, plus a spin lock added into the real SFS info structure, to be used for preventing race conditions in accessing the *used\_blocks* array in the same structure.
- *format\_real\_sfs.c*—the same file as in previous articles and the *real\_sfs* formatter application.
- *Makefile*—contains the rules for building the driver *real\_sfs\_final.ko* using the *real\_sfs\_\*.c* files, and the *format\_real\_sfs* application, using *format\_real\_sfs.c*.

With all these and the details provided earlier, Shweta completed the project's documentation. And so, finally, Shweta and Pugs were all set for their final semester

project's demo, presentation and viva. The highlights of their demo (in a root shell) were as follows:

- Loading the *real\_sfs\_final* driver: *insmod real\_sfs\_final.ko*
- Using the previously formatted pen drive partition / *dev/sdb1* or re-formatting it using the *format\_real\_sfs* application: *./format\_real\_sfs /dev/sdb1*.

**Caution:** Please check out the detailed steps for these procedures from the previous article, before you actually format it.

- Mount the *real\_sfs* formatted partition: *mount -t real\_sfs /dev/sdb1 /mnt*
- And, browsing the mounted filesystem, using the usual shell commands like *ls*, *cd*, *touch*, *vi*, *rm*, *chmod*, ...

Figure 1 shows the real SIMULA file system in action.

### Behind the scenes

And if you really want to know what enhancements Pugs added to the previous article's code to get to this level, it is basically the following core system calls, as part of the remaining 4 out of 5 sets of structures of function pointers (in *real\_sfs.c*):

- 1) *write\_inode* (under struct *super\_operations*)—*sfs\_write\_inode()* basically gets a pointer to an inode in the VFS'